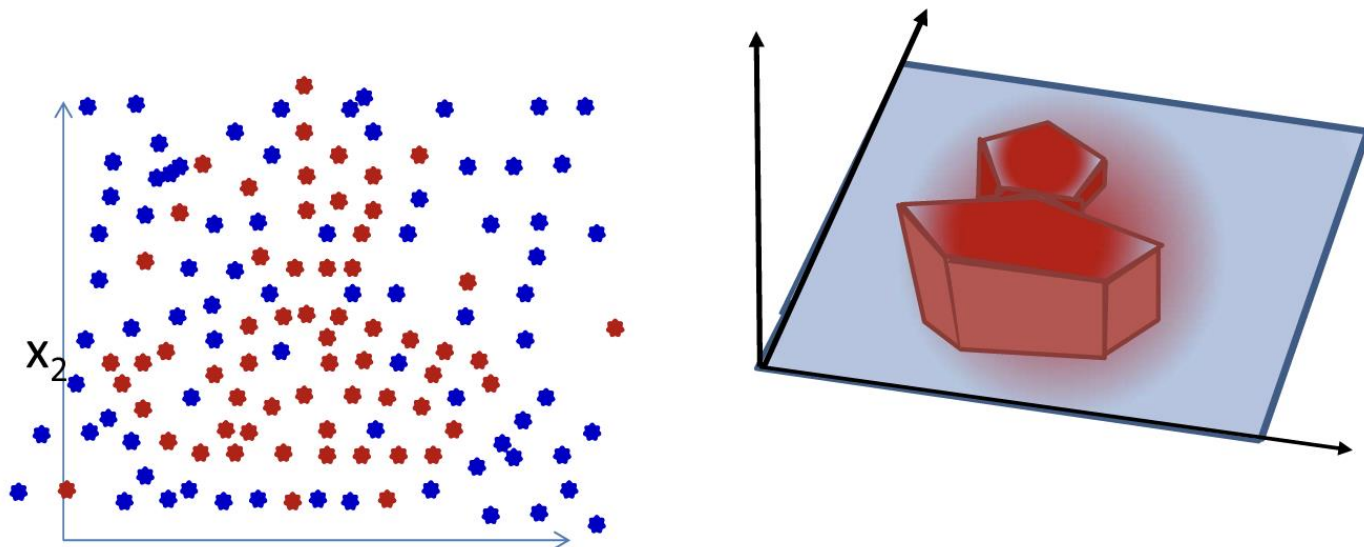


Lecture 6 MLE and Classification Evaluation

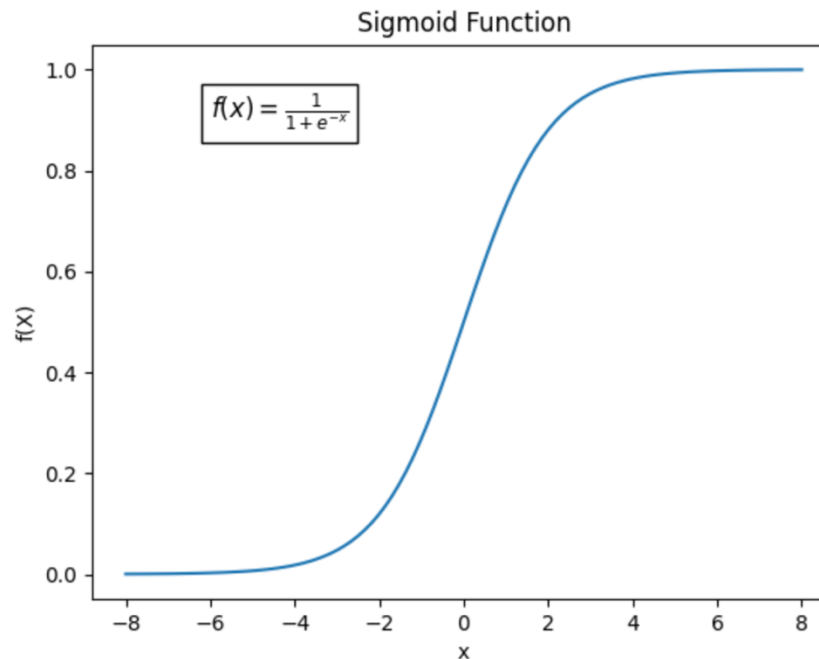
Recap: Classification VS Regression

In classification, classes are associated with regions of the feature space

- Goal: find the boundaries between these regions
- Output of classification models:
 - Represent the probabilities that a data point belongs to a given class



The Logistic Function: Properties



- The function is bounded between 0 and 1
- Tends to 1 as $z \rightarrow \infty$, and 0 as $z \rightarrow -\infty$
- Output can be interpreted as a probability

$$P_{\theta}(y = 1|x) = \sigma(\theta^T x)$$
$$P_{\theta}(y = 0|x) = 1 - \sigma(\theta^T x)$$

Recap: The Maximum Likelihood Principle

- You are given some set of observed data $\mathcal{D} = \{(X_i, y_i) | i = 1, 2, \dots, n\}$
- You choose a model $P_\theta(y|X)$ for the distribution of $y|X$
 - θ are the parameters of the model
- Estimate the θ such that $P_\theta(y|X)$ best “fits” the observations $\mathcal{D}$
 - Hoping it will also represent data outside the training set.

Defining the “Best fit”: Maximum Likelihood

- The data are generated by draws from the distribution
 - I.e., the generating process draws from the distribution
- Assumption: The world is a boring place
 - The data you have observed are very typical of the process
- Consequent assumption: The distribution has a high probability of generating the observed data
 - Not necessarily true
- Select the distribution that has the highest probability of generating the data
 - Should assign lower probability to less frequent observations and vice versa

Estimating the Model: Logistic Regression

- Given the training data $\mathcal{D} = \{(X_i, y_i) | i = 1, 2, \dots, n\}$
- Total probability of data (assuming independence)
- $P((X_1, y_1), \dots, (X_n, y_n))$

$$= \prod_{i=1}^n P(X_i, y_i)$$

Express this term in terms of the things that we know

$$= \prod_{i=1}^n P(y_i | X_i) P(X_i)$$

$$= \prod_{i=1}^n [y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] P(X_i)$$

Maximum Likelihood Estimation

- Log Likelihood

$$P((X_1, y_1), \dots, (X_n, y_n)) = \prod_{i=1}^n P(y_i | X_i) P(X_i)$$

$$\log \left(P((X_1, y_1), \dots, (X_n, y_n)) \right)$$

$$= \sum_i \log[y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] + \sum_i \log P(X_i)$$

- Maximum likelihood estimation

$$\hat{\theta} = \arg \max_{\theta} \log P((y_1, X_1), \dots, (y_n, X_n))$$

- Focusing on the bits that invoke the parameters

$$\hat{\theta} = \arg \max_{\theta} \log P((y_1 | X_1), \dots, (y_n | X_n))$$

$$\hat{\theta} = \arg \max_{\theta} \sum_i \log[y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))]$$

$$\hat{\theta} = \arg \min_{\theta} \sum_i -\log[y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))]$$

Maximum Likelihood Exercise: Coin Flip

Flipping a biased coin with unknown head probability θ

- Two possible outcomes: H or T
 - Training sample: observe 6 independent coin flips
 - $\mathcal{D} = \{x_1, x_2, x_3, x_4, x_5, x_6\} = \{H, T, H, T, T, T\}$
 - Objective: determine θ
-
- What is the first step?
 - Write down the likelihood of the data
 - $$P_{\theta}(\mathcal{D}) = \prod_{i=1}^6 P_{\theta}(x_i)$$
$$= \theta(1 - \theta)\theta(1 - \theta)(1 - \theta)(1 - \theta)$$

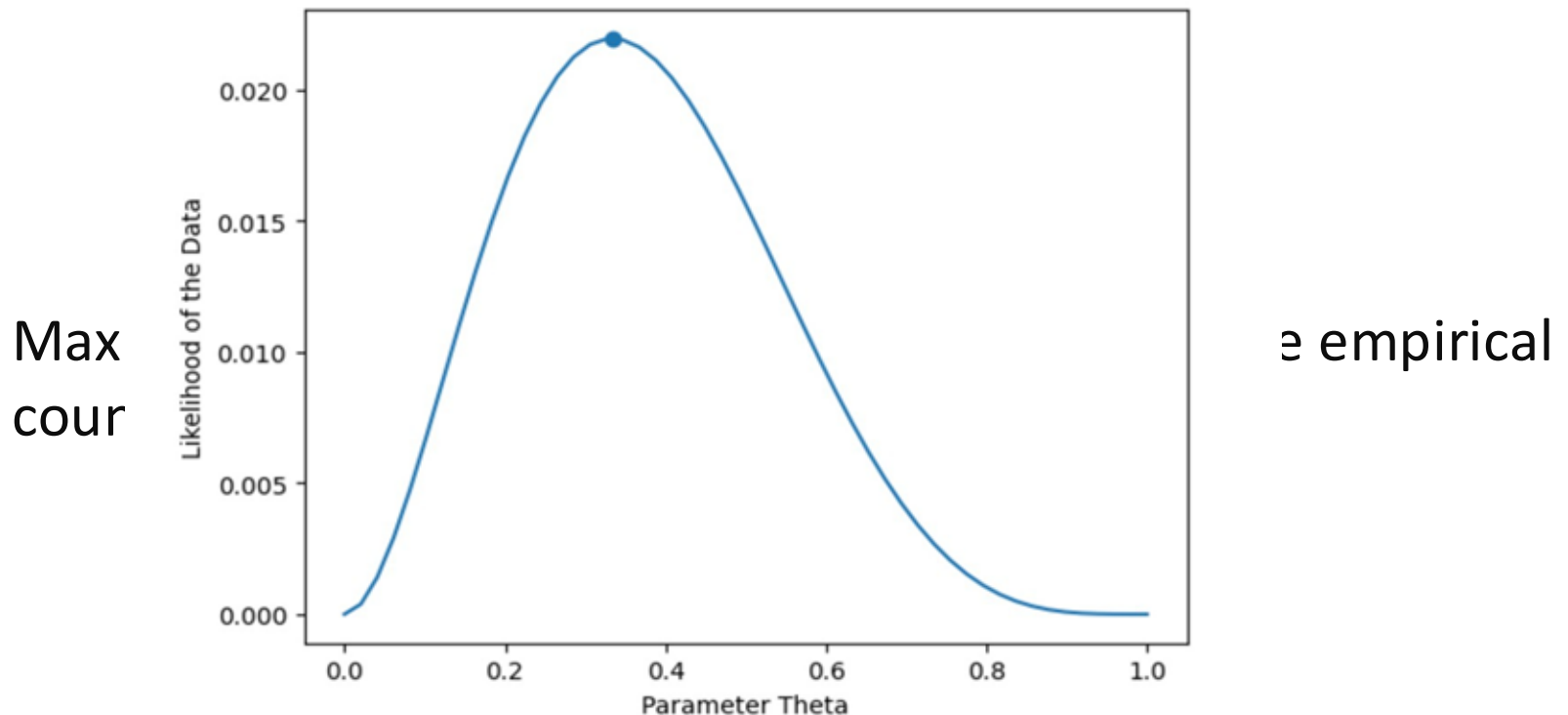
Maximum Likelihood Exercise: Coin Flip

- $$\begin{aligned} P_{\theta}(\mathcal{D}) &= \prod_{i=1}^6 P_{\theta}(x_i) \\ &= \theta(1 - \theta)\theta(1 - \theta)(1 - \theta)(1 - \theta) \\ &= \theta^{\#H}(1 - \theta)^{\#T} = \theta^2(1 - \theta)^4 \end{aligned}$$
- What is the second step?
 - Take log of the likelihood of the data
 - $\log P_{\theta}(\mathcal{D}) = \#H \log \theta + \#T \log(1 - \theta)$
- What is the third step?
 - Find the maximum of the log likelihood of the data
 - Take derivative and set it to zero

Maximum Likelihood Exercise: Coin Flip

$$\log P_{\theta}(\mathcal{D}) = \#H \log \theta + \#T \log(1 - \theta)$$

- What is the third step?
- Find the maximum of the log likelihood of the data
- Take derivative and set it to zero



Maximum Likelihood Exercise: Coin Flip

$$\log P_{\theta}(\mathcal{D}) = \#H \log \theta + \#T \log(1 - \theta)$$

- What is the third step?
- Find the maximum of the log likelihood of the data
- Take derivative and set it to zero

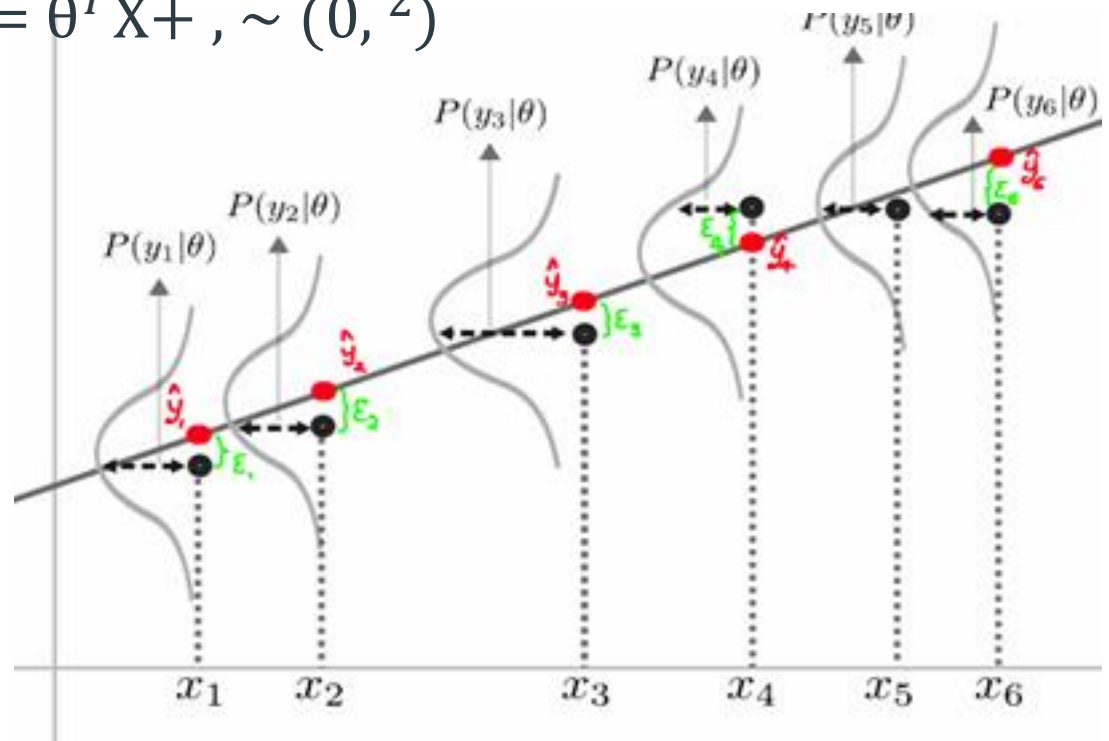
$$\frac{\partial \log P_{\theta}(\mathcal{D})}{\partial \theta} = \frac{\#H}{\theta} + \frac{\#T}{1 - \theta} = 0$$
$$\theta^* = \frac{\#H}{\#H + \#T}$$

Maximum likelihood of Bernoulli Distribution is the empirical count

Maximum Likelihood Exercise: OLS

- Recall linear regression:
- $y = \theta^T X$
- What is the likelihood of the data?
- First, make linear regression probabilistic:

$$y = \theta^T X + \epsilon, \epsilon \sim (0, \sigma^2)$$



Maximum Likelihood Exercise: OLS

- Recall linear regression:
 - $y = \theta^T X$
 - What is the likelihood of the data?
- First, make linear regression probabilistic:

$$y = \theta^T X + \epsilon, \epsilon \sim (0, \sigma^2)$$

- The density of ϵ is a Gaussian distribution:

$$P(\epsilon; \sigma) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\epsilon^2}{2\sigma^2}\right)$$

- This implies that

$$P_{\theta}(y|X) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - \theta^T X)^2}{2\sigma^2}\right)$$

Maximum Likelihood Exercise: OLS

$$P_{\theta}(y|X) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - \theta^T X)^2}{2\sigma^2}\right)$$

- Learn θ using maximum likelihood

$$-\log P_{\theta}(y|X; \mathcal{D}) = \sum_{i=1}^n c_1 \cdot (y_i - \theta^T X_i)^2 + c_2$$

- For some positive c_1
- This is the least square objective!

Maximum Likelihood Estimate

- Equals (note argmin rather than argmax)

$$\hat{\theta} = \arg \min_{\theta} \sum_i -\log[y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))]$$

- Identical to minimizing the **KL divergence** between the desired output y and actual output $\sigma(\theta^T X)$
- Maximum likelihood learning of a logistic minimizes the KL divergence between its output and the target output
- Cannot be solved directly, needs gradient descent

KL Divergence

- KL divergence: measure the distance between two distributions
- In logistic regression, the objective is to minimize the distance between the desired output y and actual output $\sigma(\theta^T X)$

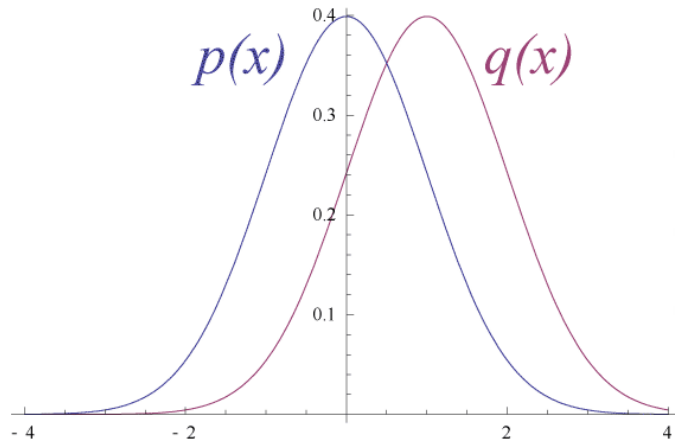
$$D(p|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

- $p(x), q(x)$: two target distributions that we are trying compare with
- Properties:
 - Non-negative; $D(p|q) = 0$ iff $p(x) = q(x)$ for all x

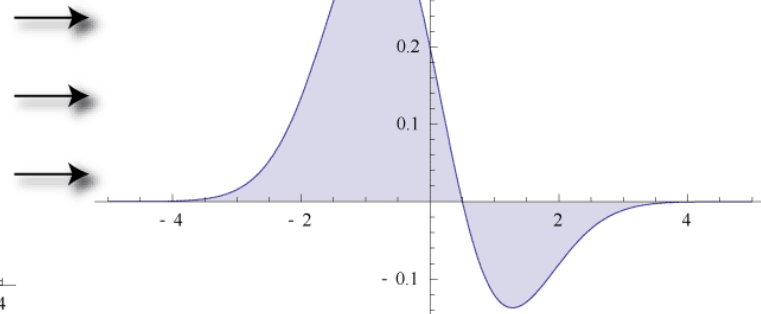
KL Divergence

$$D(p|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

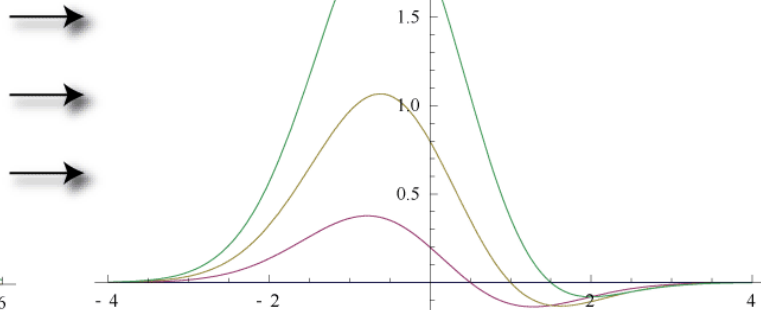
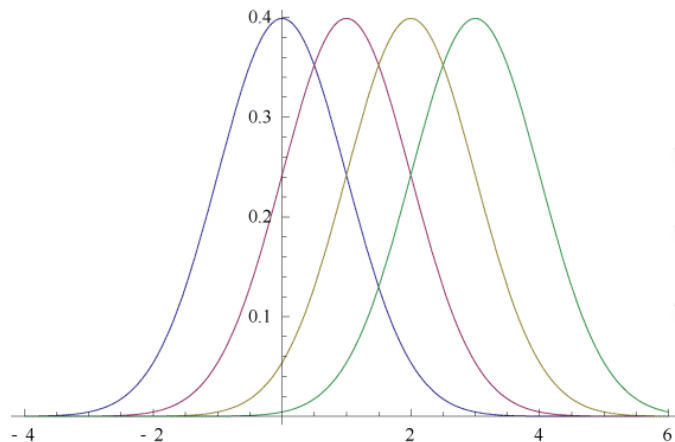
- KL divergence: measure the distance between two distributions



Original Gaussian PDF's



KL Area to be Integrated



KL Divergence & Maximum Likelihood

$$D(p|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

- $p(x) \rightarrow p(y, x)$: joint distribution of the true label and feature x .
- Constant when data is fixed
- $q(x) \rightarrow q_\theta(y, x)$: joint distribution of $\hat{y}$ given the feature x and model parameter θ

$$\arg \min_{\theta} D(p|q) = \arg \min_{\theta} \sum_x p(y, x) \log \frac{p(y, x)}{q_\theta(y, x)}$$

KL Divergence & Maximum Likelihood

$$\arg \min_{\theta} D(p|q) = \arg \min_{\theta} \sum_{x,y} p(y,x) \log \frac{p(y,x)}{q_{\theta}(y,x)}$$

$$= \arg \min_{\theta} \sum_{(y_i, X_i) \sim P_{y,x}} \frac{1}{N} (\log p(y_i, X_i) - \log q_{\theta}(y_i, X_i))$$

empirical
distribution

$$= \arg \min_{\theta} \sum_{(y_i, X_i) \sim P_{y,x}} -\log q_{\theta}(y_i, X_i)$$

possible values that x & y can take. & weigh each point by $p(y,x)$
||
sample from distribution $P(y,x)$ and take the average.

where N : # of data points.

To make the same argument for the conditional expectation $p(y|x)$, it is better to use expectations than sum $\rightarrow$ see homework

Select all correct statements

Maximum likelihood is a general principle for defining the objective/loss function of a supervised ML problem. It works beyond logistic regression

0

Maximum likelihood can always be solved directly without using gradient descent

0

When deriving the maximum likelihood, we need to write down the likelihood of data given the model parameters under the modeling assumptions

0

The goal of maximum likelihood estimation is to find the set of parameters within the chosen model class that are most likely to generate the data. We do so by maximizing the negative log-likelihood of the data

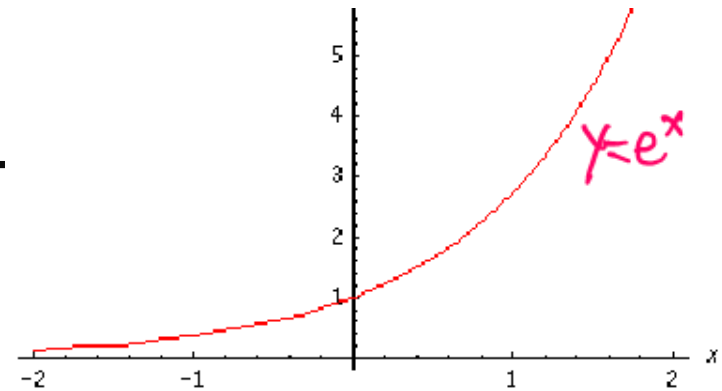
0

Recap: In Supervised Learning,

- Maximum likelihood estimation is a general principle for defining the objective/loss function.
- Maximum likelihood is based on the fact that the world is a boring place.
- Maximizing the total likelihood of the data reduces to maximizing the conditional likelihood of the label y given features X
- Maximizing the loglikelihood of the data is equivalent to minimizing the KL divergence between the true label distribution, and predicted label distribution from the model

Multiclass Classification: Softmax Regression

- The target variable contains > 2 categories
- Naïve approach: running a linear regression for each category value
- $y_1 = \theta_1^T X, y_2 = \theta_2^T X, y_3 = \theta_3^T X, \dots$
- Same issue with binary classification: the output is not guaranteed to be bounded between 0 and 1
- To make sure that the outputs are positive:
 - Pass the outcome of each linear regression to an exponential function
 - $y_1 = \exp(\theta_1^T X), y_2 = \exp(\theta_2^T X) \dots$



Multiclass Classification: Softmax Regression

- To make sure that the outputs are positive:
 - $y_1 = \exp(\theta_1^T X)$, $y_2 = \exp(\theta_2^T X)$...
- To make sure that the outputs are bounded between 0 and 1
- Normalize: $P(Y = y_i) = \frac{\exp(\theta_i^T X)}{\sum_{j=1}^K \exp(\theta_j^T X)}$
- AKA the softmax function

```
# https://scikit-learn.org/stable/auto\_examples/linear\_model/plot\_iris\_logistic.html  
from sklearn.linear_model import LogisticRegression  
logreg = LogisticRegression(C=1e5, multi_class='multinomial')
```

are the following statements correct?

0

Softmax regression generalized logistic regression from binary classification to multiclass classification

0

We can write down the objective/loss function of softmax regression as we did for logistic regression

0

We can optimize the parameters of softmax regression using gradient descent

0

Softmax Regression Generalizes Logistic Regression

Softmax regression:
$$P(Y = y_i) = \frac{\exp(\theta_i^T X)}{\sum_{j=1}^K \exp(\theta_j^T X)}$$

- Apply softmax regression to binary classification
 - Predicting $P(Y = 0)$ and $P(Y = 1)$
 - Corresponding to parameters θ_0 and θ_1
 - Reparametrization:

$$\widetilde{\theta}_0 = 0 \text{ and } \widetilde{\theta}_1 = \theta_1 - \theta_0$$

- Substitute $\widetilde{\theta}_0$ and $\widetilde{\theta}_1$ into softmax regression

$$P(Y = 1) = \frac{\exp(\widetilde{\theta}_1^T X)}{1 + \exp(\widetilde{\theta}_1^T X)} = \frac{1}{1 + \exp(-\widetilde{\theta}_1^T X)}$$

Classification Evaluation

- Output of classification algorithm $[p_{y_1}, \dots, p_{y_K}]$ with $\sum_{i=1}^K p_{y_i} = 1$
- MLE to optimize the model parameters and return a value for loss
 - This value does not tell us how well we are doing in terms of the classification task
 - First, need to convert the model output to classification prediction
- Predicted output: the most likely class $\operatorname{argmax}_i p_{y_i}$

Classification Evaluation

- Predicted output: the most likely class $\operatorname{argmax}_i p_{y_i}$
- Metric 1 **Accuracy**: whether the predicted label aligns with the true label in the dataset

$$\operatorname{acc}(f) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(f(X^{(i)}) = y^{(i)})$$

Where $\mathbb{I}(\cdot)$ is the indicator function: it equals 1 if the input is true and 0 otherwise

- When is accuracy not informative?
 - When the classes are imbalanced
 - When it is more costly to miss a positive label than a negative label

Confusion Matrix for Binary Classification

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	True positive (TP)	False Negative (FN)
Negative Class $y = 0$	False positive (FP)	True Negative (TN)

Negative or **positive** refers to the classification that our model produces

True or **False** refers to whether that classification was correct given the true label

- False positives: Type I Errors
- False negatives: Type II Errors
- $$\text{Accuracy} = \frac{TP+TN}{TP+FP+TN+FN}$$
- Or "Accuracy" on each class separately

Confusion Matrix for Binary Classification

Given a dataset of size 100, we might arrive at the following confusion matrix

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	31	20
Negative Class $y = 0$	14	35

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	True positive (TP)	False Negative (FN)
Negative Class $y = 0$	False positive (FP)	True Negative (TN)

Sensitivity and Specificity

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	True positive (TP)	False Negative (FN)
Negative Class $y = 0$	False positive (FP)	True Negative (TN)

- **Sensitivity** = $\frac{TP}{\text{positive class}} = \frac{TP}{TP+FN}$
 - AKA **recall**, true positive rate
- **Specificity** = $\frac{TN}{\text{negative class}} = \frac{TN}{FP+TN}$
 - True negative rate
- Can combine these to get a single measure called balanced accuracy
 - Balanced accuracy = $\frac{1}{2}$ (specificity + sensitivity)

Precision and Recall

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	True positive (TP)	False Negative (FN)
Negative Class $y = 0$	False positive (FP)	True Negative (TN)

- **Precision** = $\frac{TP}{\text{predicted positive}} = \frac{TP}{TP+FP}$
- Fraction predicted positive truly positive
- **Recall** = $\frac{TP}{\text{positive class}} = \frac{TP}{TP+FN}$ (True positive rate)
- Imagine we are building a search engine. The positive examples are the pages that are relevant to the users.
- Precision: of the web pages returned by the engine, how many were truly relevant?
- Recall: how many relevant pages did we find?
- Don't directly report the performance on negatives

Precision and Recall VS Sensitivity and Specificity

When do we choose precision and recall vs. sensitivity and specificity?

- Precision and recall are useful
 - When we don't care about true negatives (e.g., a search engine)
 - We only care about identifying correct positives and not missing any
- Sensitivity and specificity are useful
 - If negatives are also important (e.g., accurately detecting absence of cancer is also important)

F-Score

Harmonic mean of precision and recall

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}}$$

- Bounded between 0 and 1
- Equals to 1 at perfect precision and recall
- Equals to 0 if TP=0

Trading off Sensitivity and Specificity

Suppose that true positives are more important than true negatives

- E.g., if someone has cancer, we want to make sure we correctly identify that person
- We may output the positive class only on very confident samples
- The default threshold for predicting class 1 in binary classification is when output >0.5 . We can set it higher or lower.
- This will increase performance on positive samples and decrease performance on negative samples

Most classifiers come with “confidence scores” that make this easy to do

- Logistic regression/softmax regression: class probabilities

Receiver Operating Characteristic (ROC)

In binary classification, the ROC curve plots the **true positive rate** (TPR) and **false positive rate** (FPR) as we vary the threshold for labeling a positive sample.

- **TPR** = sensitivity = $\frac{TP}{\text{positive class}} = \frac{TP}{TP+FN}$
- **FPR** = 1 - specificity = $1 - \frac{TN}{\text{negative class}} = \frac{FP}{FP+TN}$

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	True positive (TP)	False Negative (FN)
Negative Class $y = 0$	False positive (FP)	True Negative (TN)

Receiver Operating Characteristic (ROC)

- $\text{TPR} = \text{sensitivity} = \frac{TP}{TP+FN}$; $\text{FPR} = 1 - \text{specificity} = \frac{FP}{FP+TN}$

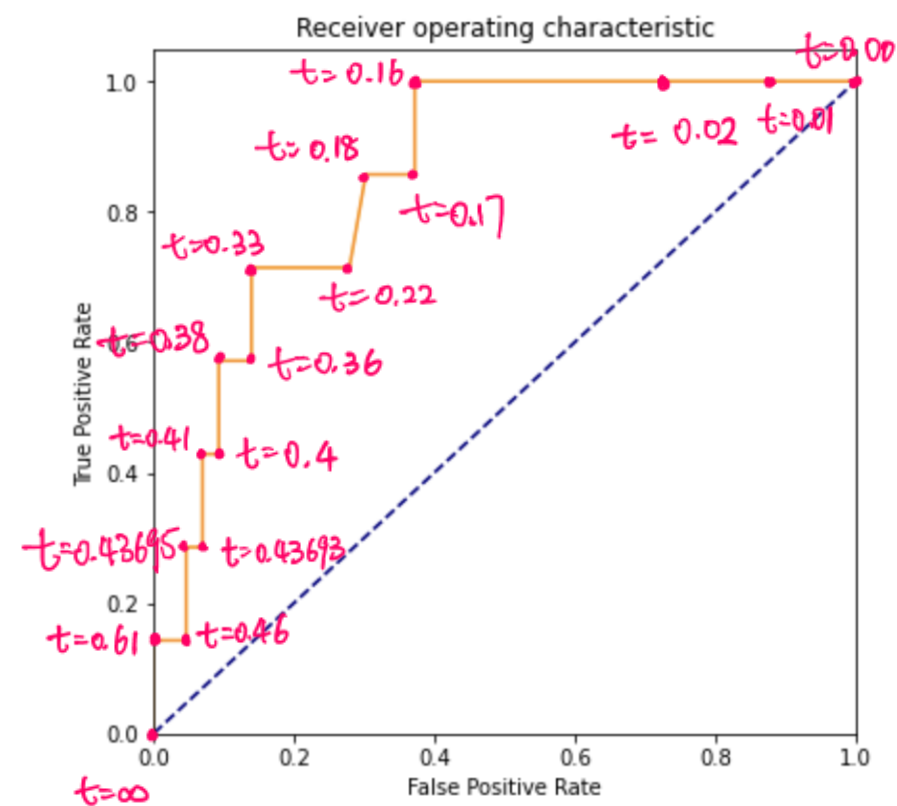
Suppose we want to improve sensitivity for Class 2 on the Iris dataset:

- We first compute the probability $P(y = 2|X)$ for each input X .
- For any threshold $t > 0$, we label X as Class 2 if $P(y = 2|X) > t$.
- Small t : e.g., identify all positive: high TPR and high FPR (many are false)
- Large t : e.g., identify few positives: low TPR and low FPR (only label the most confidence inputs)

ROC Visualization

Properties of ROC:

- Bottom left corner: predict only negatives: $TPR=FPR=0$
- Top right corner: predict only positives: $TPR=FPR=1$
- Blue diagonal:
 - Randomly guessing “positive” with $p=TPR$
- The ideal classifier
 - Top corner: $TPR=1, FPR=0$



	$\hat{y} = 1$	$\hat{y} = 0$
$y = 1$	TP	FN
$y = 0$	FP	TN

Are the following statements correct?



0

Classifiers with the same accuracy are guaranteed to have the same sensitivity and specificity

0

It is always a good idea to decompose accuracy

0

Varying the threshold of the predicted label in a classification algorithm can change the sensitivity and specificity of the model, and it does not change the parameters of the model

0

ROC curve captures trade-off between sensitivity and specificity as the threshold is varied, and it lies between ideal and random

0